R1.02 — L'Architecture Client-Serveur et le Protocole HTTP

Thème principal : Développement d'interfaces web et protocoles réseaux

Synthèse exécutive : Ce cours pose les fondements indispensables du Web en détaillant l'architecture client-serveur et le fonctionnement du protocole HTTP. Il explique comment un navigateur formule des requêtes standardisées pour obtenir des ressources, comment le serveur y répond de manière asynchrone et sans état (stateless), et comment décoder les codes de statut HTTP pour diagnostiquer et déboguer efficacement les applications web.

L'Architecture Client-Serveur et le Principe Stateless

L'architecture client-serveur est le modèle standard d'interaction sur le Web, distinguant nettement le client qui demande et le serveur qui fournit. Le protocole HTTP est qualifié de 'stateless' (sans état) : chaque échange requête-réponse est totalement indépendant. Le serveur oublie instantanément le client une fois la réponse envoyée, ce qui implique l'utilisation de mécanismes additionnels (comme les cookies ou les sessions) pour maintenir une connexion persistante ou un panier d'achat.

- Client : Logiciel actif (navigateur) qui initie la demande de ressource.
- Serveur Web : Processus logiciel passif qui écoute sur un port (ex: 80 ou 443) et traite les requêtes.
- Protocole Stateless (Sans état) : Absence de mémoire des requêtes précédentes par le protocole HTTP.

Cas concret enseignant : *Le professeur explique que lorsqu'un étudiant se connecte sur le site de l'IUT de Bayonne, son navigateur n'envoie pas une seule requête unique, mais des dizaines de requêtes HTTP simultanées pour récupérer le fichier HTML de base, puis chaque image, feuille de style CSS et script JavaScript associé.*

Structure des Requêtes, Réponses et Méthodes HTTP

Une requête HTTP se compose d'une ligne de commande, d'en-têtes et parfois d'un corps (body). La méthode GET est conçue pour la simple consultation de ressources tandis que POST est utilisée pour soumettre des données destinées à modifier l'état du serveur. En retour, le serveur renvoie une réponse structurée de manière similaire, incluant obligatoirement un code de statut à trois chiffres.

- Ligne de requête : Contient la méthode (verbe HTTP), l'URI et la version du protocole.
- En-têtes (Headers) : Métadonnées sous forme de clés-valeurs décrivant le client, l'hôte ou le contenu.
- Méthodes GET vs POST : GET est pour la lecture (paramètres dans l'URL), POST est pour la modification (paramètres dans le corps).

Cas concret enseignant : *Le professeur insiste sur le fait que le protocole HTTP est purement textuel : on peut littéralement lire les requêtes et réponses en clair si elles ne sont pas chiffrées par HTTPS. C'est pourquoi l'utilisation de GET pour transmettre un mot de passe constitue une faille majeure visible dans l'historique du navigateur.*

La Classification des Codes de Statut HTTP

Chaque réponse HTTP commence par une ligne de statut contenant un code numérique à trois chiffres permettant au client de comprendre instantanément le résultat de sa demande. Il est crucial pour un développeur de distinguer les erreurs 4xx (problème client, ex: URL fausse) des erreurs 5xx (problème serveur, ex: bug dans le script ou crash de la base de données).

- Codes 2xx (Succès) : Indiquent le succès de la requête (ex: 200 OK).
- Codes 3xx (Redirection) : Indiquent qu'une action supplémentaire est requise.
- Codes 4xx (Erreur Client) : Indiquent une anomalie dans la requête du client (ex: 404 Not Found).
- Codes 5xx (Erreur Serveur) : Indiquent une défaillance interne du serveur (ex: 500 Internal Server Error).

Cas concret enseignant : *Le professeur raconte qu'à ses débuts, il avait configuré par erreur une base de données qui plantait à chaque soumission de formulaire, renvoyant systématiquement une erreur 500. Les utilisateurs pensaient que le site était piraté alors que c'était simplement un point-virgule manquant dans son script PHP.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : Avec GET, les données du formulaire (y compris le mot de passe s'il est mal configuré) passent visiblement dans l'URL en clair, ce qui pose un problème de sécurité majeur (historique, logs). Avec POST, les données voyagent cachées dans le corps (body) de la requête HTTP, ce qui est le seul moyen acceptable pour transmettre des informations sensibles.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante et identifier la méthode utilisée, la ressource demandée, l'hôte cible, et expliquer ce que le serveur va renvoyer si la ressource n'existe pas : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Analyser la première ligne : 'GET /images/logo.png HTTP/1.1'. Le premier mot 'GET' est la méthode HTTP utilisée pour récupérer une ressource sans modifier l'état du serveur. La ressource demandée est '/images/logo.png'. 2. Identifier l'hôte cible en lisant l'en-tête 'Host: iutbayonne.univ-pau.fr'. 3. Analyser l'agent utilisateur 'User-Agent: Mozilla/5.0' qui identifie le navigateur client. 4. Déduire le comportement en cas d'absence : si le fichier logo.png n'est pas présent dans le répertoire du serveur web, celui-ci ne peut pas traiter la demande et renvoie un code de statut de la classe 4xx, précisément le code '404 Not Found'.

Résultat : Méthode : GET | Ressource : /images/logo.png | Hôte cible : iutbayonne.univ-pau.fr | Code de retour en cas d'absence : 404 Not Found (Erreur Client)

■ Auto-Évaluation Flash d'Examen

Q : Qu'est-ce qu'un protocole 'stateless' comme HTTP ?

R : C'est un protocole sans état où chaque requête est indépendante et le serveur ne conserve aucune mémoire des requêtes précédentes.

Q : Quelle est la différence majeure entre les codes d'erreur de la série 4xx et ceux de la série 5xx ?

R : Les erreurs 4xx indiquent un problème venant du client (mauvaise URL, syntaxe incorrecte), tandis que les erreurs 5xx indiquent une défaillance interne du serveur (bug de code, panne de base de données).

Q : Pourquoi n'utilise-t-on jamais la méthode GET pour transmettre un mot de passe ?

R : Parce que la méthode GET transmet les paramètres directement dans l'URL, les rendant visibles dans l'historique du navigateur, les favoris, les logs du serveur et les proxys.